Laporan Tugas 4 Pemrograman Mobile



Disusun Oleh:

Abdullah Wahid 20240801177

Dosen Pengampu:

JEFRY SUNUPURWA ASRI , S.Kom., M.Kom.

FAKULTAS ILMU KOMPUTER

UNIVERSITAS ESA UNGGUL

2026

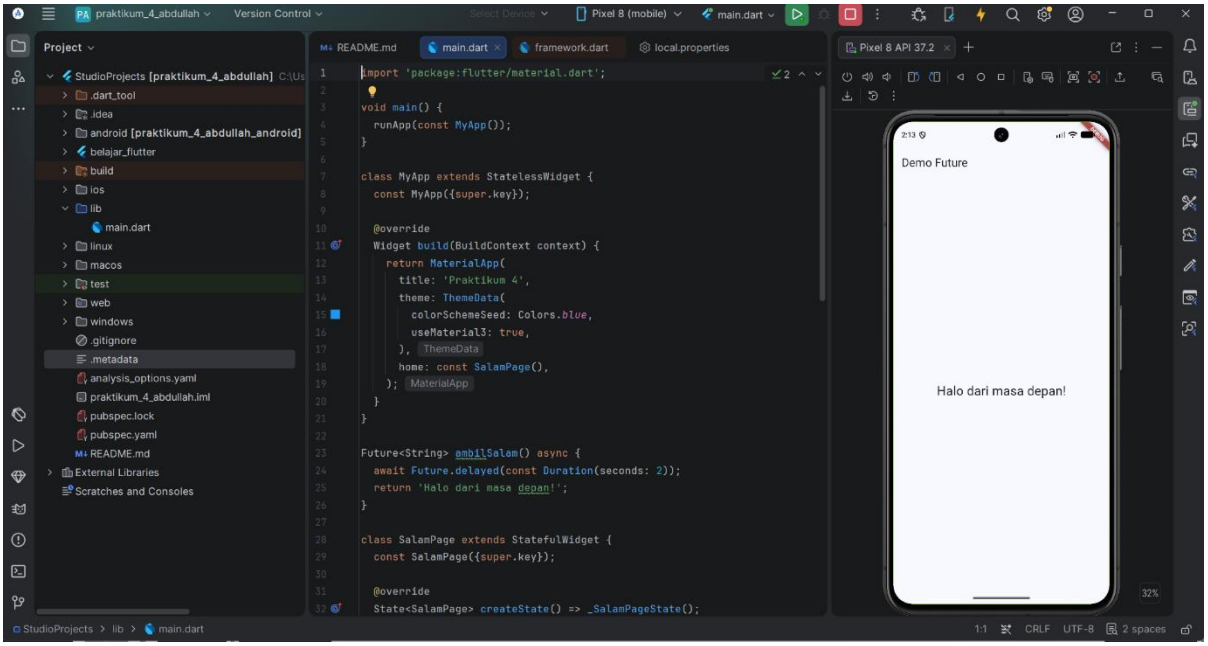


Foto 1 – Implementasi Future pada Aplikasi Flutter

Pada foto pertama ditampilkan proses pembuatan aplikasi Flutter pada project praktikum_4 dengan menggunakan file main.dart. Kode program menerapkan fungsi ambilSalam() yang memanfaatkan Future untuk menjalankan proses asynchronous. Fungsi tersebut menggunakan Future.delayed() untuk memberikan waktu tunggu selama 2 detik sebelum menampilkan pesan "Halo dari masa depan!". Pada emulator, aplikasi menampilkan halaman dengan judul "Demo Future" dan pesan salam di tengah layar setelah proses menunggu selesai. Hal ini membuktikan bahwa penerapan Future dapat digunakan untuk mengelola proses yang membutuhkan waktu secara bertahap tanpa membuat tampilan aplikasi berhenti merespons. Selain itu, program juga menggunakan MaterialApp, ThemeData, dan SalamPage untuk mengatur tampilan serta halaman utama aplikasi.

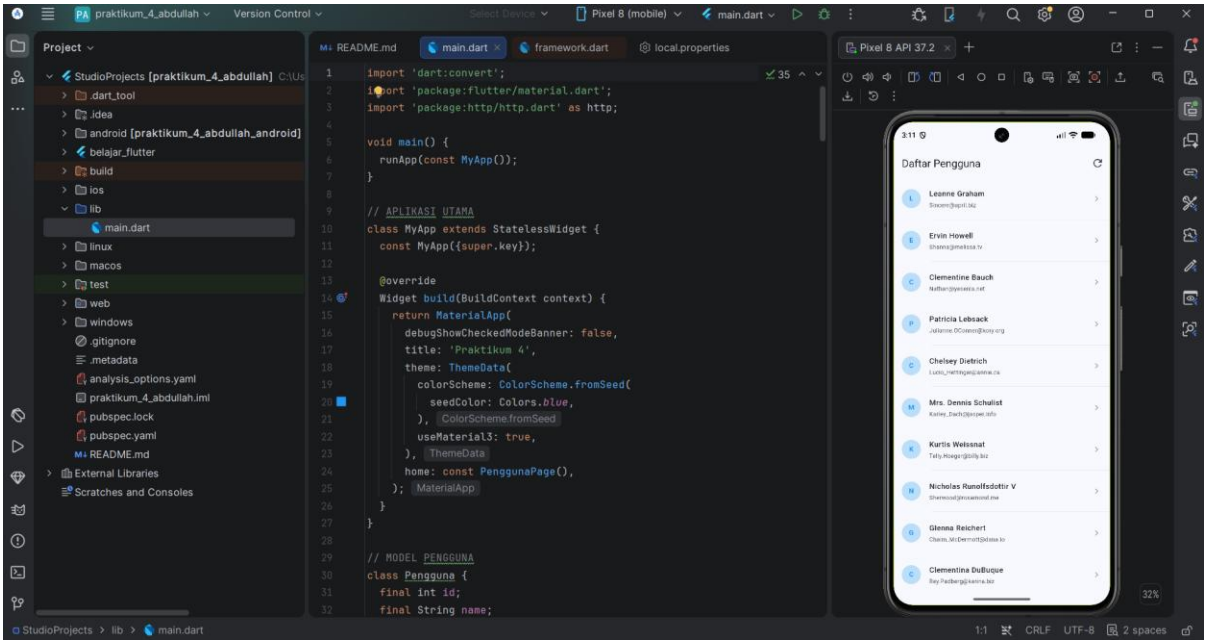


Foto 2 – Implementasi API untuk Menampilkan Data Pengguna

Pada foto kedua, aplikasi Flutter menampilkan halaman **“Daftar Pengguna”** yang memuat informasi beberapa pengguna, seperti nama lengkap dan alamat email. Setiap pengguna disajikan dalam bentuk daftar menggunakan komponen ListTile, dilengkapi avatar berbentuk lingkaran dengan huruf pertama dari nama pengguna serta ikon panah di bagian kanan. Pengguna dapat menekan salah satu item untuk membuka halaman detail yang berisi informasi tambahan.

Pada bagian program, terdapat penggunaan package http untuk mengirim permintaan ke API dan mengambil data pengguna, sedangkan dart:convert digunakan untuk memproses respons JSON menjadi objek Dart. Data yang diperoleh kemudian ditampilkan secara otomatis menggunakan komponen ListView dan FutureBuilder. Implementasi ini menunjukkan bahwa aplikasi Flutter dapat mengambil data dari sumber eksternal dan menyajikannya dalam antarmuka yang terstruktur, interaktif, dan mudah digunakan.

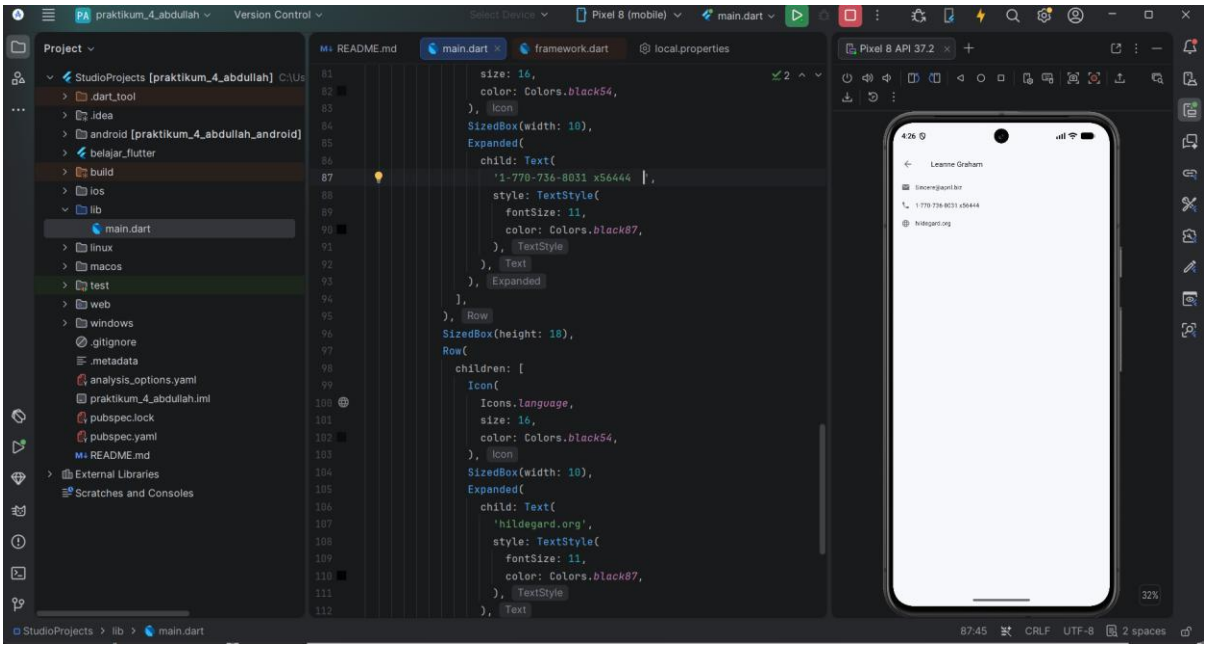


Foto 3 – Menampilkan Detail Pengguna

Pada foto ketiga terlihat halaman detail pengguna yang menampilkan nama, ID, email, nomor telepon, dan website. Data diperoleh dari API dan diubah menjadi objek Pengguna melalui parsing JSON. Navigasi menuju halaman DetailPenggunaPage menggunakan Navigator.push() dan MaterialPageRoute dengan mengirimkan data pengguna yang dipilih dari halaman daftar.

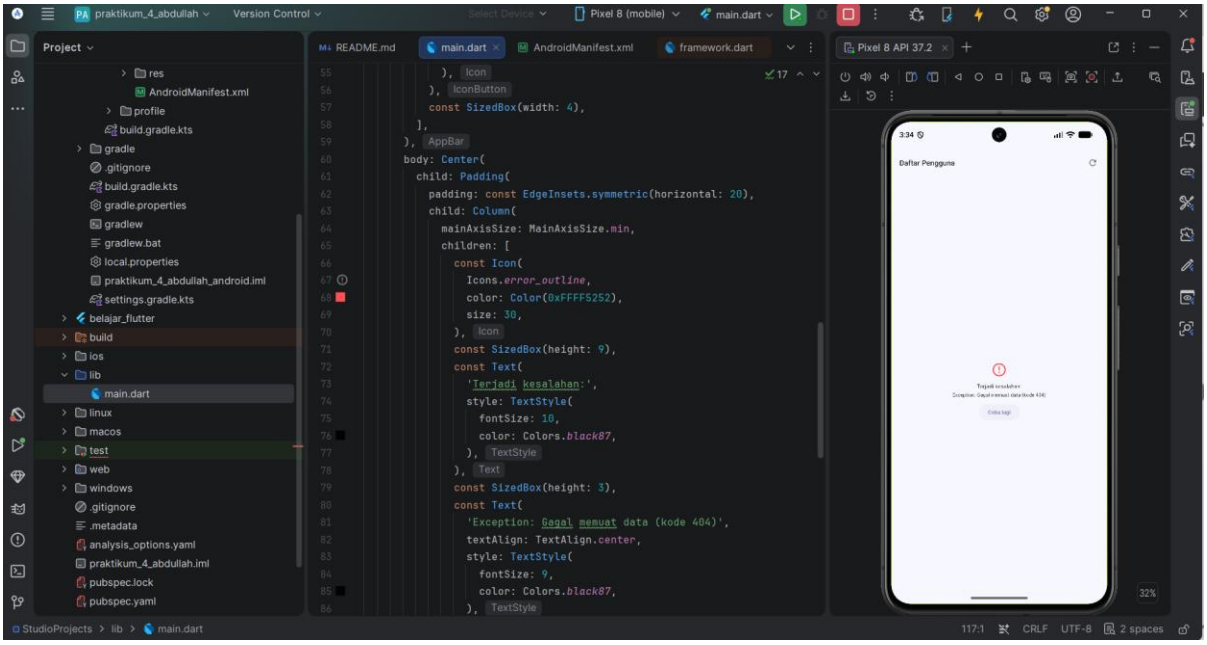


Foto 4 – Pengujian penanganan kesalahan saat mengambil data API

Pada foto keempat, aplikasi Flutter menampilkan pesan kesalahan ketika proses pengambilan data dari API gagal. Kondisi ini disebabkan oleh penggunaan endpoint yang tidak sesuai, yaitu https://jsonplaceholder.typicode.com/user. Endpoint tersebut tidak ditemukan sehingga server mengembalikan status HTTP 404. Pada program, terdapat validasi response.statusCode != 200 untuk memeriksa keberhasilan permintaan data. Jika status respons bukan 200, program akan menghasilkan exception dan menampilkan pesan kesalahan beserta tombol Coba lagi. Pengujian ini membuktikan bahwa aplikasi memiliki mekanisme error handling untuk memberikan informasi kepada pengguna ketika data tidak berhasil dimuat.

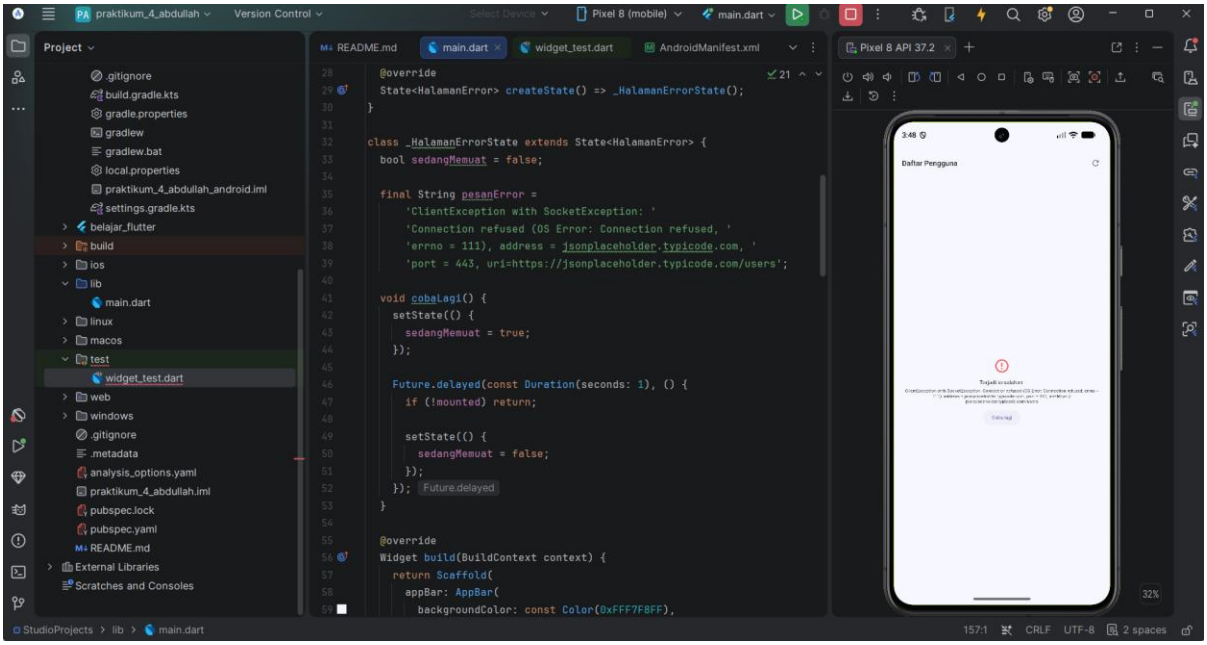


Foto 5 – Pengujian koneksi API setelah perbaikan endpoint

Pada foto kelima, alamat API telah diperbaiki menjadi https://jsonplaceholder.typicode.com/users, yang digunakan untuk mengambil data pengguna. Meskipun URL sudah sesuai, aplikasi masih mengalami kegagalan koneksi dan menampilkan pesan ClientException with SocketException: Connection refused. Hal ini menunjukkan bahwa permintaan belum berhasil mencapai server API, sehingga data tidak dapat diterima. Program menggunakan fungsi http.get() dengan batas waktu 10 detik melalui timeout() untuk membatasi durasi permintaan. Pengujian ini memperlihatkan bahwa penggunaan endpoint yang benar belum tentu menjamin pengambilan data berhasil, karena koneksi jaringan juga perlu diperhatikan. Oleh karena itu, mekanisme penanganan kesalahan diperlukan agar aplikasi dapat memberikan informasi ketika terjadi gangguan komunikasi dengan server.

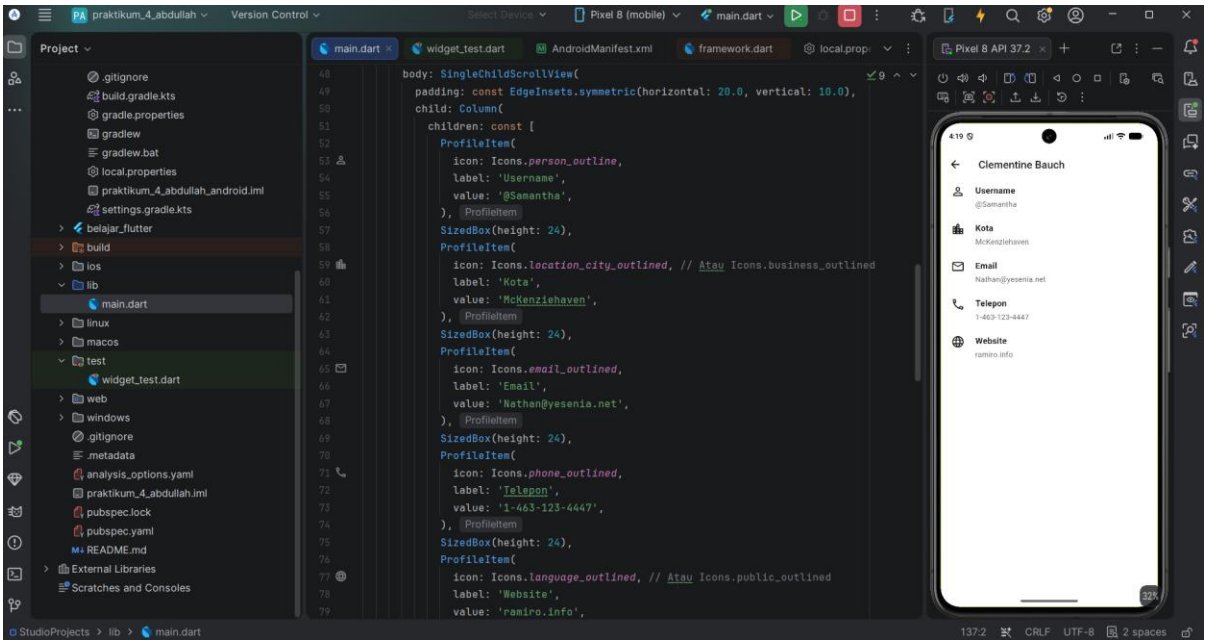


Foto 6 – Proses Parsing JSON ke dalam Model Pengguna

Pada foto keenam, terlihat implementasi fungsi Pengguna.fromJson(Map<String, dynamic> json) yang bertugas memproses data JSON yang diperoleh dari API menjadi objek Pengguna. Fungsi tersebut mengambil informasi berdasarkan key yang tersedia, seperti id, name, username, email, phone, website, dan city. Setelah data berhasil diolah, aplikasi dapat menampilkan informasi pengguna pada halaman detail, termasuk nama Clementine Bauch, username, kota, alamat email, nomor telepon, dan website. Penggunaan model Pengguna membantu menyusun data secara terstruktur sehingga setiap informasi dapat diakses dengan lebih mudah oleh komponen antarmuka. Dengan demikian, proses parsing JSON menjadi bagian penting dalam menghubungkan data dari API dengan tampilan aplikasi Flutter.